

JSON Hijacking with UTF-7

JSON Hijacking with UTF-7 - Author not stated, Publisher not stated.

- Published: date not stated
- Original: <<http://powerofcommunity.net/poc2008/hasegawa.pptx>>
- Current location: <<https://pocsec.com/poc2008/hasegawa.pptx>>
- Preserved from: <https://pocsec.com/poc2008/hasegawa.pptx> (stored) on 2026-08-09
- Capture timestamp: 20090201095000
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content (original)

The source's own words. An English translation of this document is archived beside it as `json-hijacking-utf-7_translate.md`.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

JSON Hijacking with UTF-7

--- slide 1 ---

Attacking with Character Encoding for Profit and Fun POC2008 Yosuke HASEGAWA hasegawa@utf-8.jp

Speaker notes: 1

--- slide 2 ---

Yosuke HASEGAWA NetAgent Co., Ltd R&D dept. Microsoft MVP award for Windows Security Investigating about the security issues that a character code such as Unicode causes Discovered a lot of vulnerabilities including IE and Mozilla Firefox so far, such as CVE-2008-4020, CVE-2008-0416 , CVE-2008-1468, CVE-2007-2225, CVE-2007-2227 and more... Who am I? <http://utf-8.jp/>

Speaker notes: 2

--- slide 3 ---

Introduction Comparison: match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance Invisible characters Embedded control characters Conclusion Agenda

Speaker notes: 3

--- slide 4 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 4

--- slide 5 ---

Introduction

Speaker notes: 5

--- slide 6 ---

What is the relation between charsets and security? ?

Speaker notes: 6

--- slide 7 ---

Web browser is Text Parser Handles text data such as HTML/XML... Web HTML
XML ... What's the relation between charsets and security ?

Speaker notes: 7

--- slide 8 ---

Upgrading from legacy encoding to Unicode. EUC-JP / Shift_JIS are often mixed in Unicode
Unicode EUC-JP Shift_JIS Unicode What's the relation between charsets and
security ?

Speaker notes: 8

--- slide 9 ---

Visual effect Similar lettes could be effective tools for attackers
What's the relation between charsets and security ?

Speaker notes: 9

--- slide 10 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 10

--- slide 11 ---

Comparison: match/unmatch /

Speaker notes: 11

--- slide 12 ---

String comparison and detection Basic processing for security "confirm SAFE string to pass" or "detect DANGEROUS string"

Comparison: match/unmatch

Speaker notes: 12

--- slide 13 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case

Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset

information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance

Invisible characters Embedded control characters Conclusion

Speaker notes: 13

--- slide 14 ---

Valid Invalid Overlong forms of UTF-8 One of the traditional attack techniques UTF-8

Redundant encoding / U+002F 0x2F 0xC0 0xAF 0xF0 0x80 0x80 0xAF 0xE0 0x80 0xAF

Speaker notes: 14

--- slide 15 ---

MS00-057 is famous. Currently, attacks like this have already become fossils.. IIS MS00-057

Redundant encoding

Speaker notes: 15

--- slide 16 ---

"fossils", Really? ?

Speaker notes: 16

--- slide 17 ---

CVE-2008-2938 Apache Tomcat UTF-8 Directory Traversal Vulnerability Published: Aug 12 2008 Still existing issue, not past, "Living Fossil". Redundant encoding

Speaker notes: 17

--- slide 18 ---

Countermeasure: Don't implement functions handling UTF-8 yourself. Convert all strings into UTF-16 beforehand UTF-8 UTF-16 Redundant encoding

Speaker notes: 18

--- slide 19 ---

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 19

--- slide 20 ---

Conversions from Unicode to others has several "many-to-one" pairs. Unicode

Many-to-one Conversion U+005C ¥ U+00A5 \ 0x5C U+20A9 In Japan, path delimiter is
displayed as YEN-SIGN.

Speaker notes: 20

--- slide 21 ---

Many-to-one Conversion Input string as Unicode Validation Processing ¥..¥..¥ Bypass
filtering Path traversal U+00A5 U+005C Convert to other encodings

Speaker notes: 21

--- slide 22 ---

".." and "..\..\Windows" is existing in "C:\temp" folder. Path traversal occurs when handling
filenames as ANSI. ANSI Many-to-one Conversion

Speaker notes: 22

--- slide 23 ---

Many-to-one Conversion DEMO

Speaker notes: 23

--- slide 24 ---

A lot of letters converted from Unicode are "many-to-one". Many-to-one Conversion U+00A1
U+00A6 U+00C0 U+00C1 U+00C2 U+00C3 À ¡ ¨ Á Â Ã U+00C4 U+00C5 Ä Å U+00C6 Æ 0x41 A 0xA5
0x7C ! |

Speaker notes: 24

--- slide 25 ---

Countermeasure: Handle strings as Unicode, without conversion. Don't convert after validation,
even if conversion is necessary. Unicode () Many-to-one Conversion

Speaker notes: 25

--- slide 26 ---

Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance Invisible characters Embedded control characters Conclusion

Speaker notes: 26

--- slide 27 ---

Definition of the identification for Upper-Case and Lower-Case is different by a language culture.
Upper case and Lower case

Speaker notes: 27

--- slide 28 ---

Comparison of Upper-Case and Lower-Case Upper case and Lower case Word Equivalent
Nonequivalent Gif / GIF U.S. Turkey Ma ße/ MASSE Germany U.S. Ma ße /
Masse Switzerland Germany U.S. Windows , ISBN978-4-
7561-5000-4,P.340

Speaker notes: 28

--- slide 29 ---

Countermeasure: Don't adopt difference between lower case and upper case as boundary of security. Never rely on case-conversion rules you expect.
Upper case and Lower case

Speaker notes: 29

--- slide 30 ---

Agenda / Introduction Comparison:
7 match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance Invisible characters Embedded control characters Conclusion

Speaker notes: 30

--- slide 31 ---

Unicode supports the Composition and Decomposition of letters. No differences in appearance, but byte sequences are different Unicode Normalization
U+304C U+304B U+3099 Precomposed character Base character Combining Character

Speaker notes: 31

--- slide 32 ---

Unicode defines four specific forms of normalization. NFC Normalization Form Canonical Composition NFD Normalization Form Canonical Decomposition NFKC Normalization Form Compatibility Composition NFKD Normalization Form Compatibility Decomposition Cannot restore original byte sequence after Normalization. Unicode 4
Normalization

Speaker notes: 32

--- slide 33 ---

Normalization process changes the byte sequence into another of different meaning
Normalization U+2025 U+002E . U+002E . U+2473 U+0031 1 NFKC,NFKD

Speaker notes: 33

--- slide 34 ---

Normalization Input string as Unicode Validation Normalization Processing ¥ ¥ ¥
Bypass filtering Path traversal U+2025 U+002E

Speaker notes: 34

--- slide 35 ---

Countermeasure: Never normalize strings after validation. Normalization

Speaker notes: 35

--- slide 36 ---

Agenda / Introduction Comparison:
7 match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 36

--- slide 37 ---

Depending on the implementation, illegal byte sequence is often ignored or converted to
unexpected characters. Embedded invalid
characters

Speaker notes: 37

--- slide 38 ---

Firefox prior to 2.0.0.12 had ignored 0x80 under Shift_JIS encoding. Firefox 2.0.0.12
Shift_JIS 0x80 Embedded invalid characters <s [0x80] c [0x80] r [0x80] ipt> alert(1) </s
[0x80] c [0x80] r [0x80] ipt>

Speaker notes: 38

--- slide 39 ---

IE ignores 0x00. IE 0x00 Embedded invalid characters <s [0x00] c [0x00] r [0x00] ipt>
alert(1) </s [0x00] c [0x00] r [0x00] ipt>

Speaker notes: 39

--- slide 40 ---

IE considers 0x0B and 0x0C as delimiter. IE 0x0B 0x0C Embedded invalid
characters <script [0x0B]

alert(1) </s cr ipt> <input type=text value= a [0x0C] onmouseover=alert(1)

Speaker notes: 40

--- slide 41 ---

Countermeasure: Generate only safe string with white listing.
Embedded invalid characters

Speaker notes: 41

--- slide 42 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 42

--- slide 43 ---

Inject leading byte of Multi Byte Character Set(MBCS) to bypass filters
Embedded leading bytes

Speaker notes: 43

--- slide 44 ---

Invalidate quotation with 0x82, leading byte of Shift_JIS. Shift_JIS 0x82
Embedded leading bytes name: <input type=text value=" [0x82]"> e-mail: <input type=text
value= " onmouseover=... //">

Speaker notes: 44

--- slide 45 ---

Bypass XSS Filter of IE8 using leadbyte of MBCS. IE8 XSS Filter Embedded leading bytes UTF-
8 http://example.com/?%3cscript%20 %E2%3E alert(1);... http://example.com/? %E2%22
onmouseover=alert(1) Shift_JIS http://example.com/?%3cscript%20 %81%3E %3ealert(1);... EUC-JP
http://example.com/?%3cscript%20 %E0%3E alert(1);... http://example.com/? %E0%22
onmouseover=alert(1)

Speaker notes: 45

--- slide 46 ---

Countermeasure: Validate by a letter unit. Convert another encoding...
Embedded leading bytes

...

Speaker notes: 46

--- slide 47 ---

Agenda / 7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 47

--- slide 48 ---

Different understanding about the charset between server and client charset
Mismatch in charset information <html> < > Process Escape Generate HTML User
1101100110010 0010001110110 1000010100110 0101011011110 < <

> " " & & ' ' UTF-8 UTF-7

Speaker notes: 48

--- slide 49 ---

Typical issue is XSS with UTF-7 When charset is ambiguous, IE assumes it as UTF-7 and causes XSS.
UTF-7 XSS charset IE UTF-7 XSS Mismatch in charset
information

Speaker notes: 49

--- slide 50 ---

No charset is specified neither HTTP response header nor <meta> charset Mismatch
in charset information HTTP/1.1 200 OK Content-Type: text/html ... <html><head> <meta http-
equiv="content-type" content=" text/html "> </head><body> +ADw-script+AD4- alert(1)
+ADw-/script+AD4- ...

Speaker notes: 50

--- slide 51 ---

Unrecognizable charset name for IE IE charset Typically wrong charset names are:
CP932 / MS932 / sjis / jis / utf8 ... Mismatch in charset information <meta http-equiv='content-type'
content='text/html; charset=CP932 '> +ADw-script+AD4- alert(document.cookie);
+ADw-/script+AD4-

Speaker notes: 51

--- slide 52 ---

Unrecognizable charset name for IE Google, Yahoo, IBM ... IE doesn't recognize "CP932", "CP950",
"EUC" for charset name Mismatch in charset information http://www.google.com/search?
oe=CP932 &q= %2bADw- ... http :// www.google.com/search? oe=CP950 &q = % 2bADw- ... http ://
search.yahoo.com/search? eo=EUC &p= %2bADw- ...

Speaker notes: 52

Inject fake <meta> before original it. <meta> <meta> Mismatch in charset information

```
<title> +ADw-/title+AD4- +ADw-meta http-equiv+AD0-'content-type' content+AD0-'text/html+ADs-charset+AD0-utf-7'+AD4- </title> <meta http-equiv='content-type' content='text/html; charset= euc-jp '>
```

Combination of UTF-7 with Ignoring Content-Type of IE. IE6 doesn't support "application/atom+xml" for Content-Type. Determine as UTF-7 HTML by content. Mismatch in charset information HTTP/1.1 200 OK Content-Type : application/atom+xml <?xml version='1.0' encoding= 'utf-8 '?>... <title>Search: +ADw-/title+AD4- +ADw-script+AD4- ... No charset

Speaker notes: 55

Speaker notes: 56

Speaker notes: 57

Speaker notes: 58

JSON Hijacking with UTF-7 Mismatch in charset information [{ "name" : " abc"}];var t=[{"":" ", "mail" : "hasegawa@utf-8.jp" }, { "name" : "Kanatoko", "mail" : "anvil@example.com" }] JSON for target: http://example.com/target.json No charset in HTTP response header

Speaker notes: 59

--- slide 60 ---

JSON Hijacking with UTF-7 Mismatch in charset information <script src="http://example.com/target.json" charset=" utf-7 "></script> <script> alert(t[1].name + t[1].mail); </script> Trap page: [{ "name": " abc"}];var t=[{"": " ", "mail": "hasegawa@utf-8.jp" }, { "name": "Kanatoko", "mail": "anvil@example.com" }] JSON UTF-7 setter

Specify charset as UTF-7 from outside of JSON. No need to use **defineSetter**

Speaker notes: 60

--- slide 61 ---

Mismatch in charset information DEMO

Speaker notes: 61

--- slide 62 ---

Countermeasure for JSON: Place "while (1);" before JSON text. Accept only "POST", Reject access by "GET".

	while(1);	JSON	POST	Mismatch in charset information
1	while(1);	JSON	POST	Mismatch in charset information

Speaker notes: 62

--- slide 63 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 63

--- slide 64 ---

IE ignores the most significant bit of US-ASCII. IE US-ASCII Interpreting 7-bit
encoding 0010 0010 2 2 " 0x22 1 010 0010 A 2 0xA2 0011 1100 3 C < 0x3C 1 011 1100 B C 0xBC
0011 1110 3 E

0x3E 0xBE 1 011 1110 B E

Speaker notes: 64

--- slide 65 ---

Interpreting 7-bit encoding

Speaker notes: 65

--- slide 66 ---

MIME-Version: 1.0 Content-Type: text/plain; charset=US-ASCII Content-Transfer-Encoding: 7bit This is test mail begin 644 eicar.com #5/(5 E0\$%06S1<4%l8-30H4%Xl-T-#*3=1) \$5)0T%2+5-404Y\$05)\$+4%.

75\$E625)54RU415-4+49)3\$4A)\$@K2"!# end OE also ignores the most significant bit of US-ASCII .

Interpreting 7-bit encoding 0xCD M 0x4D 0xB6 6 0x36

Speaker notes: 66

--- slide 67 ---

Countermeasure: Specify charset clearly on HTTP response header. Don't use US-ASCII. Use ISO-8859-1 and so on. HTTP charset US-ASCII ISO-8859-1 Interpreting 7-bit encoding

Speaker notes: 67

--- slide 68 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance Invisible characters Embedded control characters Conclusion

Speaker notes: 68

--- slide 69 ---

Deceptive indications

Speaker notes: 69

--- slide 70 ---

Visual effect for human being Provoke a mistake Effective and useful tool for attackers
Deceptive indications

Speaker notes: 70

--- slide 71 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance Invisible characters Embedded control characters Conclusion

Speaker notes: 71

--- slide 72 ---

Such as " 1 " (Digit One) and " l " (Small letter L)... [http://bank 1 .example.com/](http://bank1.example.com/) <http://bank l .example.com/> More and more on Unicode ... 1 () l () Unicode
Characters with similar appearance

Speaker notes: 72

--- slide 73 ---

Solidus and Division Slash Characters with similar appearance `http:// example.co.jp /`
`t.example.com /foo/bar` Domain name / U+2215 / U+002F Solidus Division Slash

Speaker notes: 73

--- slide 74 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 74

--- slide 75 ---

Invisible byte sequence Unicode ISO-2022-JP Escape sequences Invisible characters U+200B ZERO
WIDTH SPACE U+200C ZERO WIDTH NON-JOINER U+200D ZERO WIDTH JOINER U+202A LEFT-TO-
RIGHT EMBEDDING U+FEFF BYTE ORDER MARK (ZWNBS) 0x1B 0x24 0x40 0x1B 0x24 0x42 0x1B
0x28 0x42

Speaker notes: 75

--- slide 76 ---

Using for filename, registry Invisible characters

Speaker notes: 76

--- slide 77 ---

Invisible characters DEMO

Speaker notes: 77

--- slide 78 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 78

--- slide 79 ---

Unicode Bidirection (Bidi) Part of string is displayed from RIGHT to LEFT U+202E (Right-to-Left
Override;RLO) Unicode (Bidi) Embedded control characters this-
(U+202E) txt.exe this-exe.txt Actual byte sequence Displayed text

Speaker notes: 79

--- slide 80 ---

Embedded control characters this- (U+202E) txt.exe this-exe.txt Actual byte sequence Displayed text

Speaker notes: 80

--- slide 81 ---

Embedded control characters DEMO

Speaker notes: 81

--- slide 82 ---

Countermeasure: Prepare multiple confirmation methods SSL / EVSSL Display as Punycode
Deceptive indications

Speaker notes: 82

--- slide 83 ---

Agenda /

7

Introduction Comparison:

match/unmatch Redundant encoding Many-to-one Conversion Upper case and Lower case
Normalization Embedded invalid characters Embedded leading bytes Mismatch in charset
information Interpreting 7-bit encoding Deceptive indications Characters with similar appearance
Invisible characters Embedded control characters Conclusion

Speaker notes: 83

--- slide 84 ---

Conclusion

Speaker notes: 84

--- slide 85 ---

Never convert to another encoding or normalize after validating strings. Don't be deceived only by an appearance. Security issues concerning character encodings are uncultivated fields.

×

Conclusion

Speaker notes: 85

--- slide 86 ---

Yosuke HASEGAWA hasegawa@netagent.co.jp hasegawa@utf-8.jp http://utf-8.jp/ Questions?

Speaker notes: 86